

Aqui está o relatório abrangente da investigação tecnológica e arquitetural para o **Hermes Creative Workstation**. A pesquisa foi conduzida com rigor de engenharia, avaliando repositórios reais, licenças, viabilidade técnica de longo prazo e integrações com o ecossistema existente do Hermes Agent (Laya/System-1, Task Compiler, Experience Compiler).

Parte I — Mapa Tecnológico

Um panorama das tecnologias mais adequadas para cada domínio criativo, filtradas por maturidade e compatibilidade com arquiteturas web/Electron.

1. Design 2D e Edição Vetorial

- **Penpot** (`penpot/penpot`): O padrão ouro open source (MPL 2.0). Renderiza via SVG/DOM. Escrito em ClojureScript. Excelente para UI/UX, mas SVG sofre gargalos críticos de performance em animações a 60fps.
- **Konva** (`konvajs/konva`): Framework 2D Canvas (MIT). Altíssimo desempenho para manipulação de milhares de objetos. Ideal para fundações de editores gráficos performáticos.
- **Fabric.js / PixiJS**: Fabric (Canvas) é maduro para manipulação de objetos; PixiJS (WebGL) é imbatível para renderização 2D de alta performance, mas exige que a interface vetorial seja construída do zero.

2. Motion Graphics e Animação

- **Theatre.js** (`theatre-js/theatre`): Excepcional (Apache 2.0). Um editor visual de animação (dope sheet, graph editor) totalmente agnóstico de renderizador. Pode animar DOM, Canvas ou Three.js.
- **Remotion** (`remotion-dev/remotion`): Muito popular (62k+ stars), permite criar vídeos via React. **Atenção à Licença**: Gratuito apenas para uso não comercial ou empresas de até 3 pessoas; uso em batch exige licença paga.
- **Revideo** (`midrender/revideo`): Motor de renderização alternativo em TypeScript focado em criação programática de vídeo com código (MIT).

3. Criação, Edição e Animação 3D

- **React Three Fiber (R3F)** (`pmndrs/react-three-fiber`): Wrapper declarativo (MIT) do Three.js para React. Integrável facilmente em interfaces complexas e altamente compatível com Theatre.js.
- **Three.js Editor / Babylon.js**: São robustos, mas funcionam melhor como engines standalone do que como módulos embutidos em uma timeline NLE genérica.

4. Edição e Processamento de Vídeo (NLE)

- **OpenCut** (`OpenCut-app/OpenCut`): O grande destaque recente (87k+ stars, MIT). Alternativa ao CapCut com core reescrito em **Rust/WASM**, interface Next.js, arquitetura de plugins e — o mais crítico — um **Servidor MCP nativo para Agentes de IA**.
- **OpenReel** (`Augani/openreel-video`): Alternativa 100% web escrita em TypeScript (MIT). Possui integração VModal, sem necessidade de servidores externos. Excelente UI, mas o core de compositing em JS é menos escalável que o Rust do OpenCut.
- **WebCodecs / FFmpeg.wasm**: A base de decodificação no browser/Electron para qualquer solução moderna.

5. Recursos de IA e Visão Computacional

- Modelos locais executáveis via ONNX/WebNN (para Background Removal com RMBG-1.4, Whisper para legendagem).
- **Integração nativa via MCP** (Model Context Protocol): O próprio motor da aplicação expõe operações para o *Experience Compiler* do Hermes.

Parte II — Repositórios e Forks Relevantes

Projeto / Repositório	Licença	Maturidade	Vantagens	Limitações	Audiência
OpenCut (<code>OpenCut-app/OpenCut</code>)	MIT	Alta (em reescrita)	Core em Rust/WASM, arquitetura plugin-first, servidor MCP nativo, modo headless.	Desktop nativo (GPUI) ainda em desenvolvimento; migração em curso.	Sim (canc... princi...
OpenReel (<code>Augani/openreel-video</code>)	MIT	Média-Alta	Completamente web, dependências limpas, boa UI de timeline.	Performance de compositing limitada pela engine JS pura.	Sim
Penpot (<code>penpot/penpot</code>)	MPL 2.0	Altíssima	Modelo de componentes e tipografia excepcionais.	Baseado em Clojure e SVG. Difícil mesclar com engines WebGL/Vídeo.	Não o cor (garg de st
Penpot Fork (<code>girafic/penpot</code>)	MPL 2.0	Experimental	Branch <code>girafic-penpot-timeline-</code>	Renderização ainda no DOM/SVG.	Avalia com de UI

Projeto / Repositório	Licença	Maturidade	Vantagens	Limitações	Audi
			animation prova a viabilidade de keyframes no Penpot.	Inadequado para exportação NLE real.	
Theatre.js (<code>theatre-js/theatre</code>)	Apache 2	Alta	Interface de keyframes e graph editor pronta e desacoplada do renderizador.	É uma biblioteca de animação, não um editor de vetores.	Sim (canc a mó

Parte III — Análise Arquitetural

A arquitetura para o **Hermes Creative Workstation** não pode ser uma "colcha de retalhos", mas também não deve ser recriada do zero. O Hermes Work tem como base Electron/React e um backend Python. O *Task Compiler* e o *System-1* precisam de controle programático (JSON/RPC/MCP) do documento.

- **A. Adaptar um software existente (Penpot): *Rejeitada*.** Modificar o Penpot introduz a barreira do ClojureScript. Mais crítico: a árvore DOM/SVG do Penpot não suporta compositing de vídeo e WebGL a 60fps de forma síncrona.
- **B. Criar editor próprio: *Rejeitada*.** Escrever do zero timelines, decodificação WebCodecs e graph editors leva anos de engenharia especializada.
- **C. Arquitetura Híbrida Baseada em NLE (Vencedora):** Utilizar o **OpenCut** como *host application/container* estrutural. O OpenCut fornece o gerenciamento de projeto, a renderização Rust/WASM e a timeline NLE. Dentro deste host, integram-se motores especializados:
 - **Theatre.js** atuando como controlador de estado no tempo (para gráficos complexos 2D e 3D).
 - **React Three Fiber / Konva** embutidos em WebGL Contexts que são enviados como *streams/layers* para o motor de compositing do OpenCut.

Essa abordagem preserva o conceito de documento único (a composição de vídeo) e estende as capacidades de cada "camada" (Layer) para ser um documento vetorial ou cena 3D editável.

Parte IV — Auditoria dos Finalistas (Foco no Hermes)

1. OpenCut (A Base Core)

- **Arquitetura:** React (UI) + Rust (WASM/Compositing).
- **Modelo de Documento:** Representado via JSON serializável, perfeito para LLMs manipularem diretamente (atendendo ao `HERMES_WORKSTATION_INTELLIGENCE.md`).
- **Integração IA:** Já possui um **Servidor MCP** documentado que expõe as APIs do editor. O *Laya/System-1* pode literalmente "conversar" com a timeline, solicitar cortes, adicionar assets e mudar textos.

2. Theatre.js (O Motor de Animação Fina)

- **Papel:** Fornece o "Studio" visual para curvas de Bézier e interpolação.
- **Editabilidade Bidirecional:** É notório por sincronizar perfeitamente o código (ou JSON gerado por IA) e a interface humana. Se a IA altera um valor (ex: `scale: 1.5`), o humano vê o *keyframe* atualizado na hora, e vice-versa.

Parte V — Comparação Técnica (Matriz de Decisão)

Critério	Peso	Híbrida (OpenCut + Theatre.js + R3F)	Adaptação (Penpot + girafic fork)	Construção Customizada (Zero)	OpenRe
Funcionalidades prontas	25%	9	6	2	8
Integração IA (MCP/JSON)	15%	10 (Nativo)	4	8	5
Unificação / Editabilidade	20%	8	9	10	6
Facilidade de implementação	15%	7	3	1	8
Extensibilidade/Arquitetura	10%	9	5	9	6
Estabilidade e manutenção	10%	8	3	2	7
Licenciamento (MIT/Apache)	5%	10 (MIT)	7 (MPL 2.0)	10	10
SCORE FINAL PONDERADO	100%	8.60	5.25	4.90	7.00

Justificativa do vencedor: A abordagem Híbrida via OpenCut maximiza as funcionalidades base de edição pesada (Rust/WASM) enquanto a abstração em React facilita embutir plugins 3D e 2D. O fork experimental do Penpot esbarra no limite do SVG para motion, e o Remotion falha no critério de licença aberta para empresas.

Parte VI — Recomendação Final (As 10 Respostas)

1. **Qual projeto é mais vantajoso?** O **OpenCut** (nova versão Rust/React). Ele já resolve a fundação de vídeo, UI de edição não-linear, exportação e integração de IA via protocolo MCP.
2. **Existe fork superior?** O próprio rewrite arquitetural do OpenCut em Rust atua como a evolução que procurávamos. O fork `girafic-penpot-timeline-animation` é valioso apenas como estudo de UX, mas tecnicamente inadequado para nossa stack.
3. **Quais bibliotecas adicionar?** **Theatre.js** para animação granular/graph editor e **React Three Fiber (R3F)** para o canvas 3D.
4. **Melhor estratégia de IA?** Utilizar o **Servidor MCP** já embutido no OpenCut. O *Task Compiler* do Hermes atua como cliente MCP, despachando intenções (ex: "remova fundos e aplique um zoom"). O modelo visual permanece unificado, permitindo que o humano faça fine-tuning visual da mesma árvore de estado.
5. **Como preservar um único arquivo editável?** O documento universal deve ser o projeto do OpenCut (um descritor JSON multitrack). Camadas estáticas 2D e composições 3D são abstraídas como "Track Items" específicos, contendo seu próprio `state_tree` que motores embutidos (Konva, R3F) renderizam.
6. **O que já existe vs O que desenvolver?** *Existe:* Timeline, decode/encode, compositing Rust, protocolo de IA (MCP). *Desenvolver:* O plugin vetor/2D avançado dentro do OpenCut (substituindo textos simples por camadas ricas do Konva) e o plugin 3D.
7. **Menor trabalho para resultado utilizável?** Embutir o OpenCut clássico via iframe ou componente React no Hermes Desktop Workstation, expondo imediatamente edição de vídeo acelerada.
8. **Potencial longo prazo?** A separação entre Core (Rust) e UI (React) garante sobrevida técnica, compatibilidade Web/Desktop (Electron) e alta performance com WebGPU no futuro.
9. **O que NÃO vale a pena desenvolver?** Um motor de renderização de vídeo e um gestor de memória em WebAssembly (já resolvido pelo OpenCut/WebCodecs), e um visualizador de keyframes (resolvido pelo Theatre.js).
10. **O que deve ser comprovado?** Testar se a comunicação (binding) entre os valores do Theatre.js e as instâncias de objetos no motor Rust do OpenCut consegue rodar a 60fps (Teste 2 e Teste 4) sem overhead extremo na bridge React/WASM.

Parte VII — Plano de Implementação

Respeitando os contratos de upstream/downstream da branch `workstation` no repositório `hermes-agent`.

Plano de Resultado Imediato (Semanas 1–4)

Foco: Entregar um editor de vídeo utilizável e acoplado ao Sistema 1 do Hermes.

1. **Fork/Vendor:** Integrar o OpenCut web na arquitetura do Hermes Desktop.
2. **Setup MCP:** Conectar o `hermes-acp` (Agente) ao servidor MCP exposto pelo OpenCut.
3. **Teste 3 (Vídeo) e Teste 5 (IA):** Validar importação de mídia, cortes e permitir que o *Task Compiler* gere edições estruturais a partir de prompts. Exportar projeto.
4. **Critério de sucesso:** O usuário solicita edições de vídeo por texto, a IA corta o vídeo, e o usuário pode abrir a timeline e ajustar os cortes manualmente.

Plano de Evolução (Meses 2–5)

Foco: Transformar de "Editor de Vídeo" para "Creative Workstation Universal".

1. **Integração Theatre.js:** Plugar a interface do Theatre.js para permitir animações granulares e interpolações complexas que o OpenCut base não possui (Teste 2).
2. **Camadas 3D (R3F):** Criar um tipo de camada customizada no OpenCut que renderize uma cena `React Three Fiber`. Implementar bridge para que o motor Rust trate o canvas 3D como uma fonte de textura em tempo real (Teste 4).
3. **Módulo de Design 2D:** Incorporar Konva para permitir que objetos dentro da camada de vídeo sejam grupos vetoriais avançados e editáveis (Teste 1).
4. **Inteligência Visão:** Embarcar WebNN localmente (Background Removal, Auto-tracking) diretamente no pipeline de ferramentas de máscara do ambiente.
5. **Critério de sucesso (Teste 6):** Salvar um arquivo contendo vídeo raw, uma camada vetorial complexa, e um objeto 3D animado por IA. Fechar o Hermes, reabrir e poder alterar a cor do objeto 3D via interface visual humana sem perda de propriedades.